Arv, grunder

Arv, omdefiniering av metoder (override)

Abstrakta klasser och metoder

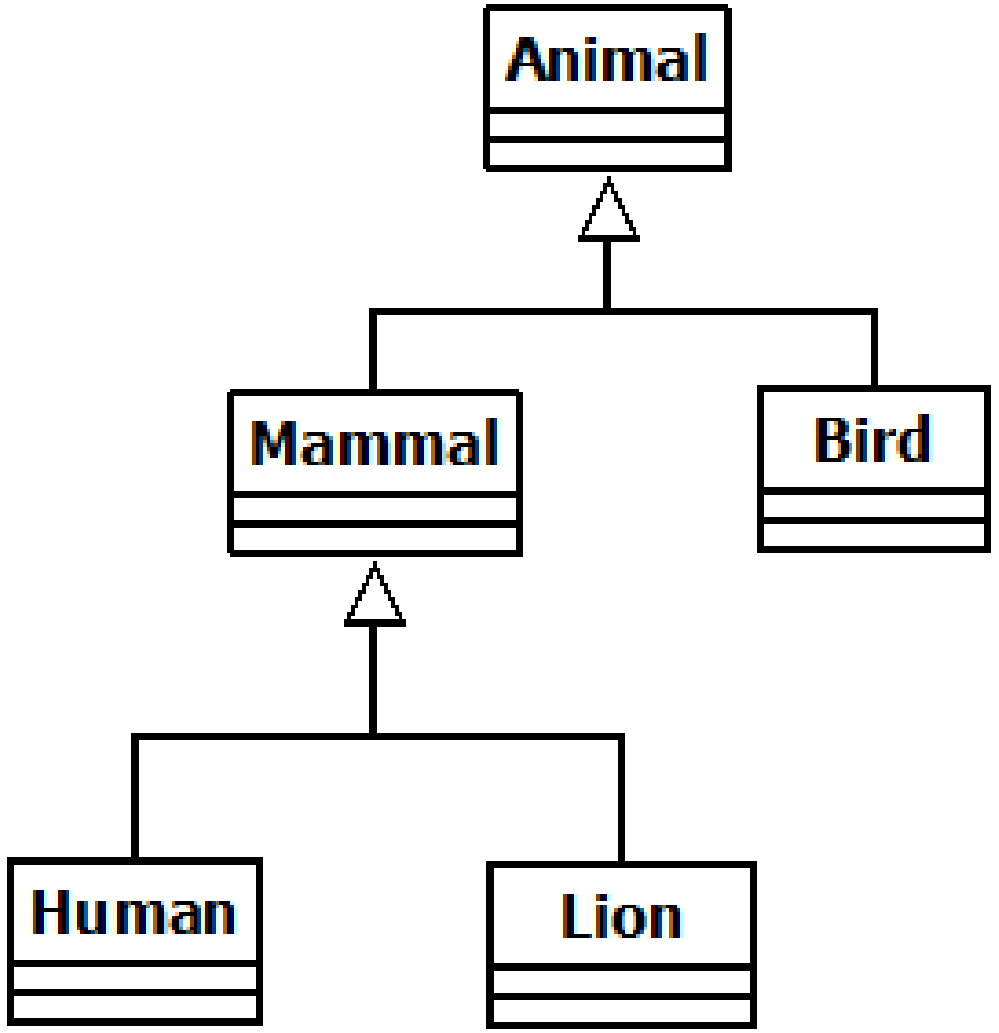
Objektorienterad programmering

- Abstrakta datatyper – definierade via klasser
  - Modellera verkliga eller abstrakta objekt
  - Definierar möjliga tillstånd och beteende för objekten
  - Inkapsling – ”hide the implementation”
  - Programmera mot klassens publika gränssnitt
  - Återanvändning
- Hög kohesion – varje klass modeller en och endast en sak
- Låg koppling (lågt beroende) mellan klasser
- Arv
  - Super- och subtyper
  - Polymorfism

Arv (inheritance)

- *Arv är en relation mellan klasser (typer)*
  - Till skillnad från association och aggregat, som är relationer mellan objekt
- En klass kan *ärva* datamedlemmar och metoder från en annan klass
  - Subklassen har då samma egenskaper som superklassen, men vi kan lägga till ytterligare
- Andra namn på arv: specialisering/generalisering

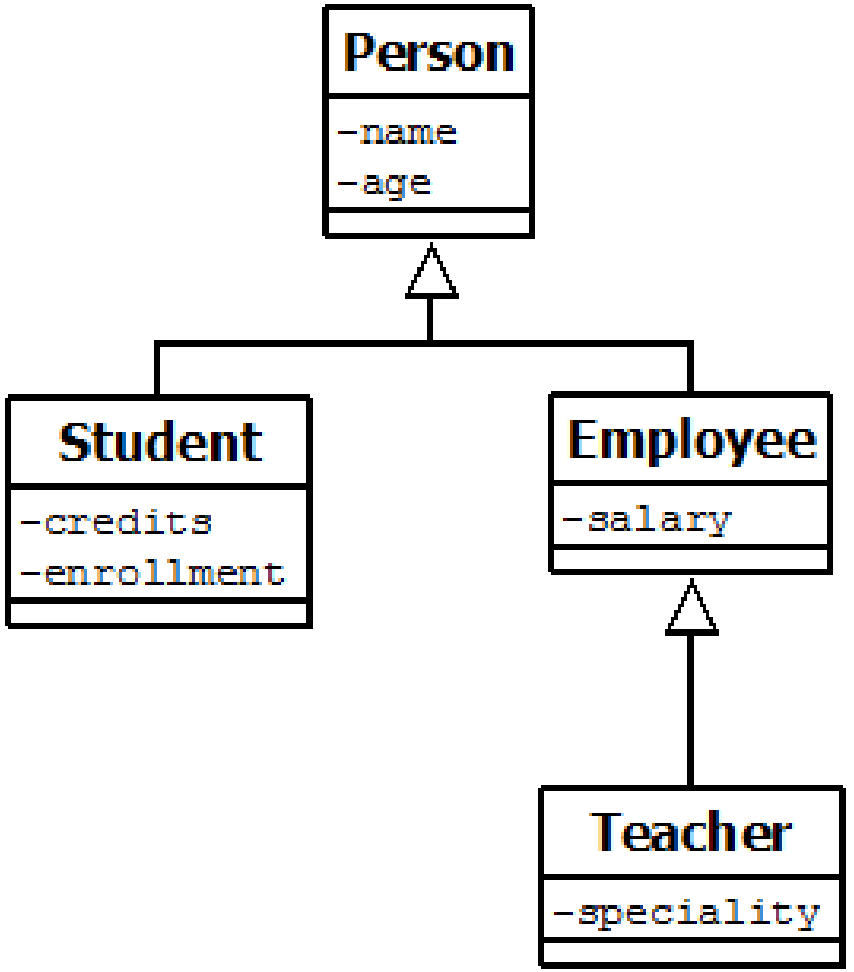
Arv, ”är en”



generalisering

specialisering

Arv, ”är en”



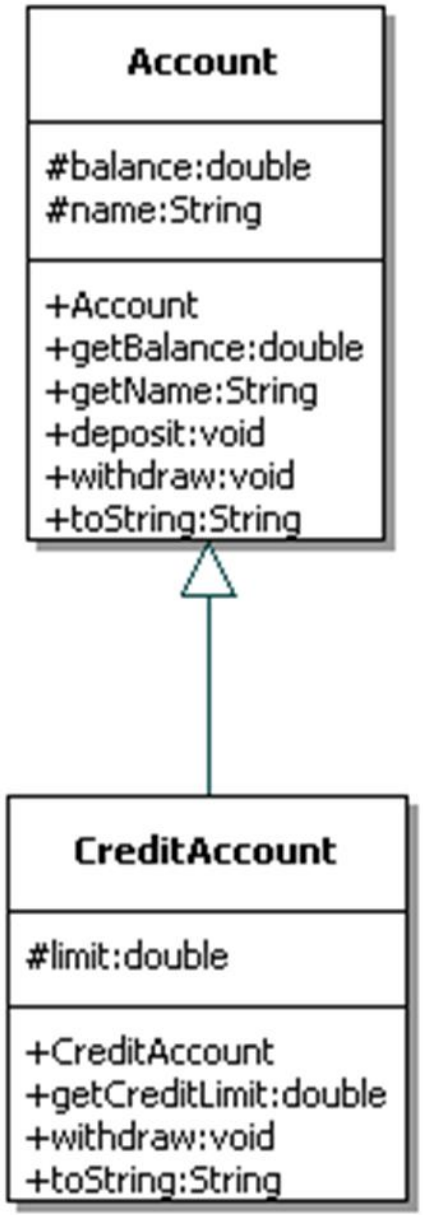
Ett objekt av typen Teacher har datamedlemmarna name, age, salary och speciality

Vad ärver subklassen

- Alla datamedlemmar ärvs, *inklusive private* och static, men…
  - Datamedlemmar som är privata i superklassen är inte ”synliga”, dvs. kan inte direkt manipuleras, i subklassen …
  - … men de existerar i objekten
- Alla metoder ärvs, utom privata metoder *och konstruktorer*
- Subklassens konstruktor ska alltid(!) anropa superklassens konstruktor, super(args…) …
  - … på första raden i subklassens konstruktor
- Nyckelordet ”super” refererar, i en subklass, till metod/datamedlem/konstruktor i superklassen

Arv

Relation mellan *klasser*, ”är en”



Super-klassen – en helt vanlig klass:

```
public class Account {

    protected double balance;
    protected String name;
    …
```

Sub-klassen, utökad med en begränsad kredit:

```
public class CreditAccount extends Account {

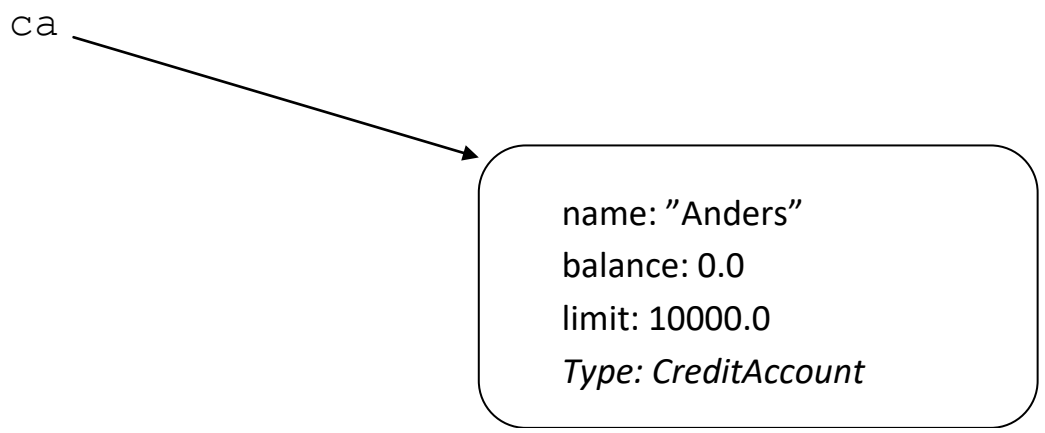
    protected double limit;

    public CreditAccount(String name, double limit) {
       super(name);
        this.limit = limit;
    }
    …
```

Arv, ”är en”

Referens och objekt skapas på vanligt sätt

```
CreditAccount ca = new CreditAccount("Anders", 10000.0);
```



Synlighet, visibility, i Java

Keyword/ UML-notation	Tillgänglig inom klassen?	Tillgänglig inom samma paket?	Tillgänglig i subklass?	Tillgänglig för alla andra klasser?
private -	Ja	Nej	Nej	Nej
“package private” ~	Ja	Ja	Nej	Nej
protected #	Ja	Ja	Ja	Nej
public +	Ja	Ja	Ja	Ja

Omdefiniera en ärvd metod – ”override”

- Logiskt sett samma operation som i superklassen, men ska utföras något annorlunda
- withdraw(…) i CreditAccount måste ta hänsyn till kreditgränsen, limit
- Metodens signatur måste var identisk i subklassen: *returtyp*, namn, parameterlista

```
public class CreditAccount extends Account {

    protected double limit; // Credit limit
    …

    public double getCreditLimit() { return limit; }
    …

    @Override
    public void withdraw(double amount) {
       if(amount > balance + limit || amount < 0) {
           throw new IllegalArgumentException(amount);
       }
       else {
          balance -= amount;
       }
    }
    …
}
```

Omdefiniera en ärvd metod (override)

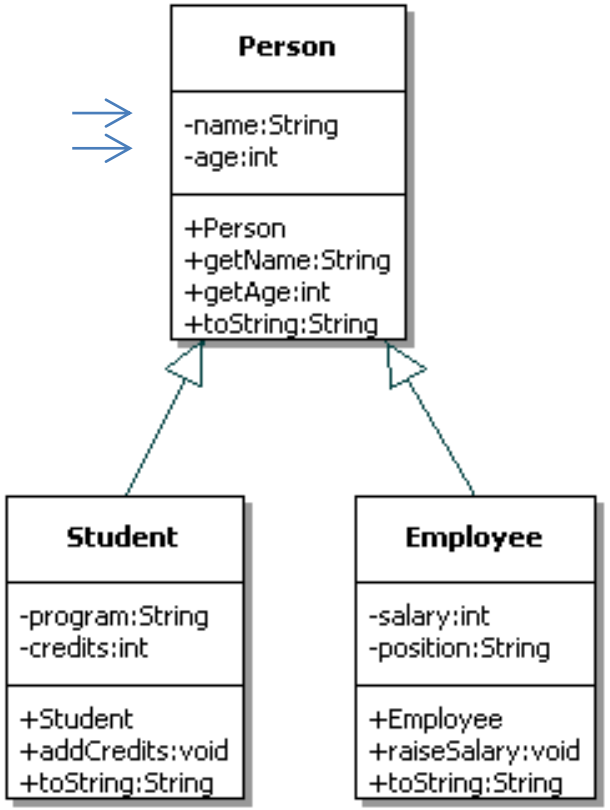
- Med annotationen ”@Override” kontrollerar kompilatorn om signaturen är korrekt,
  dvs om det finns en metod med identisk signatur i superklassen
- Skilj omdefiniering av metod vid arv, override, från överlagring av metoder inom en klass, overload (samma metodnamn, men olika argumentlistor, i en klass)

```
public class CreditAccount extends Account {
    …

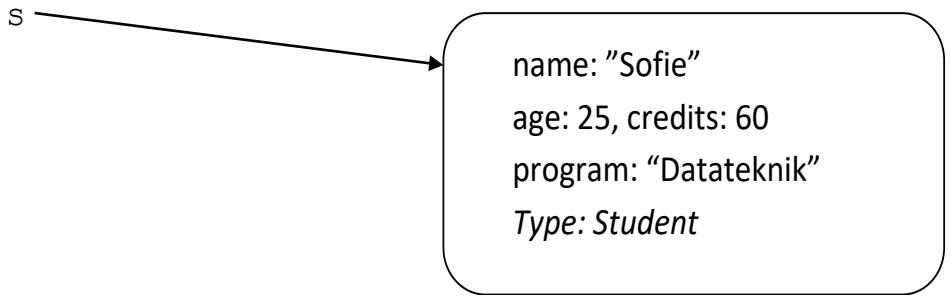
    @Override
    public void withdraw(double amount) {
        if(amount > balance + limit || amount < 0) {
            throw new IllegalArgumentException(amount);
        }
        else {
            balance -= amount;
        }
    }
    …

    @Override
    public String toString() {
        String info = super.toString();
        info += ", credit limit: " + limit;
        return info;
    }
}
```

Privat data vid arv




```
Student s =
    new Student("Sofie",25,"Datateknik",60);
```



Privat data vid arv

- Ärvda datamedlemmar med synligheten private kan ej refereras direkt i koden för subklassen
- Vi kan använda ärvda metoder om de är deklarerade public eller protected i superklassen

```
public class Student extends Person {

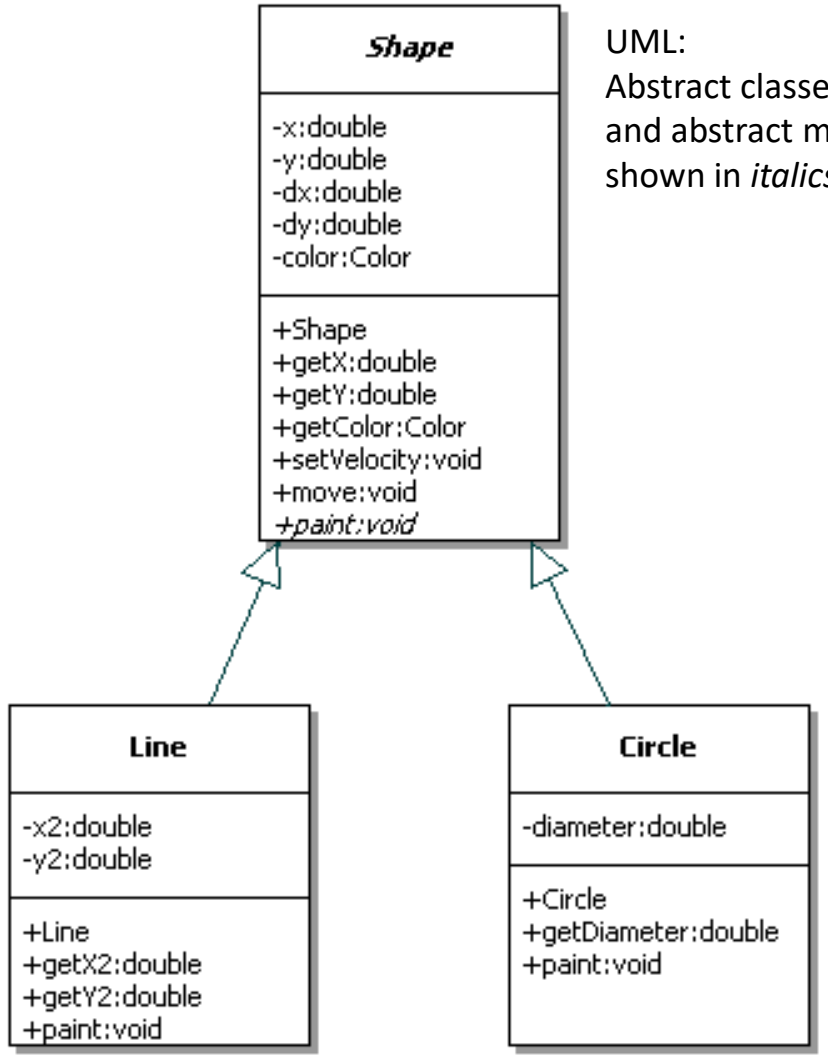
    private String program;
    private int credits;
    // name, age are inherited, but private in super class

    public Student(
          String name, int age, String program, int credits) {
       super(name, age );
       this.program = program;
       this.credits = credits;
    }
    …

    @Override
    public String toString() {
       String info = this.getName() // use a public method
       + ", program  " + program
       +", age  " + this.getAge()
       + ", " + credits + " credits";
       return info;
    }
}
```

Abstrakta klasser och metoder

- Ett exempel: Vi behöver klasser som representerar ritbara figurer i ett ritprogram, tex Line och Circle
- Klasserna Line och Circle har vissa gemensamma egenskaper – vi vill slippa dessa i båda klasserna
- Lösning: Flytta det gemensamma, datamedlemmar och metoder till en superklass, Shape - generalisering!
- Men:
  - Vi vill inte att det ska gå att skapa objekt av typen Shape
  - Det kan finnas metoder som bör deklareras i superklassen, men inte kan ges implementation förrän i subklasserna (här: paint)



UML:
Abstract classes, interfaces and abstract methods are shown in *italics*

Abstrakta klasser och metoder

- En abstrakt klass får datamedlemmar och metoder, men …
- … man kan inte skapa objekt av abstrakt typ
  - Konstruktor behövs dock för subklasser
- En abstrakt klass får ha abstrakta metoder, dvs metod-deklarationer utan implementation
- Subklasser måste implementera ärvda abstrakta metoder

```
abstract public class Shape {

    private double x, y;
    private double dx, dy;

    protected Shape(double x, double y) {
       this.x = x; this.y = y;
    }

    public double getX() { return x; }
    public double getY() { return y; }
    …

    public void move() {
       x += dx; y += dy;
    }

    abstract public void paint();
}
```

Subklass till abstrakt klass

- Använd den abstrakta klassens konstruktor i subklassens konstruktor …
  - … super(args …)
- Alla ärvda abstrakta metoder måste implementeras
  - Räknas som en omdefiniering
  - Om inte alla abstrakta metoder implementeras blir även subklassen abstrakt

```
public class Circle extends Shape {

    private double diameter;

    public Circle(double x, double y, double diameter) {
       super(x, y);
       this.diameter = diameter;
    }

    public double getDiameter() { return diameter; }

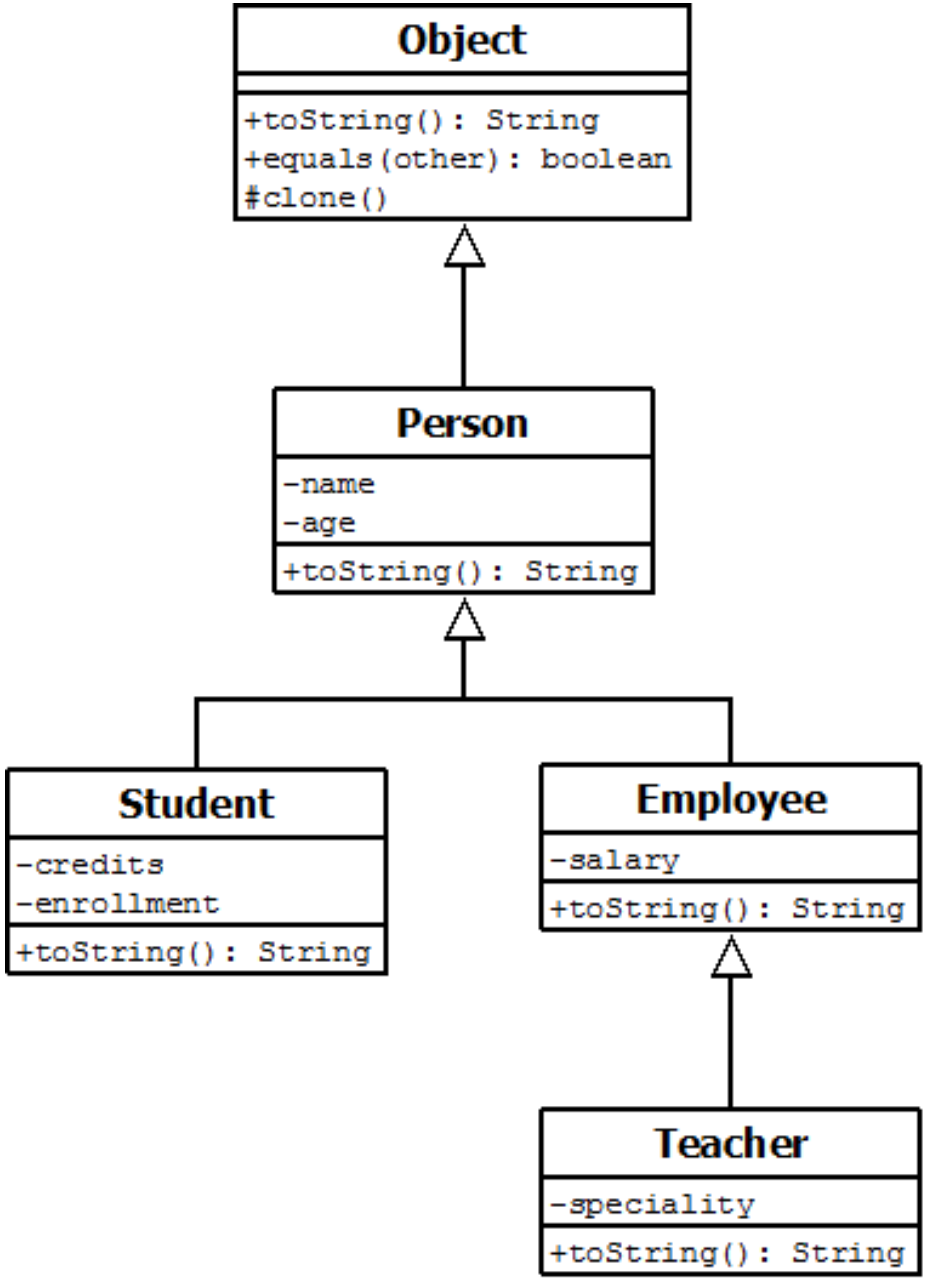
    @Override
    public void paint() {
       System.out.println("This is a circle at (" +
          getX() + "," + getY() + "), diameter " +
          diameter);
    }
}
```

Java och superklassen Object

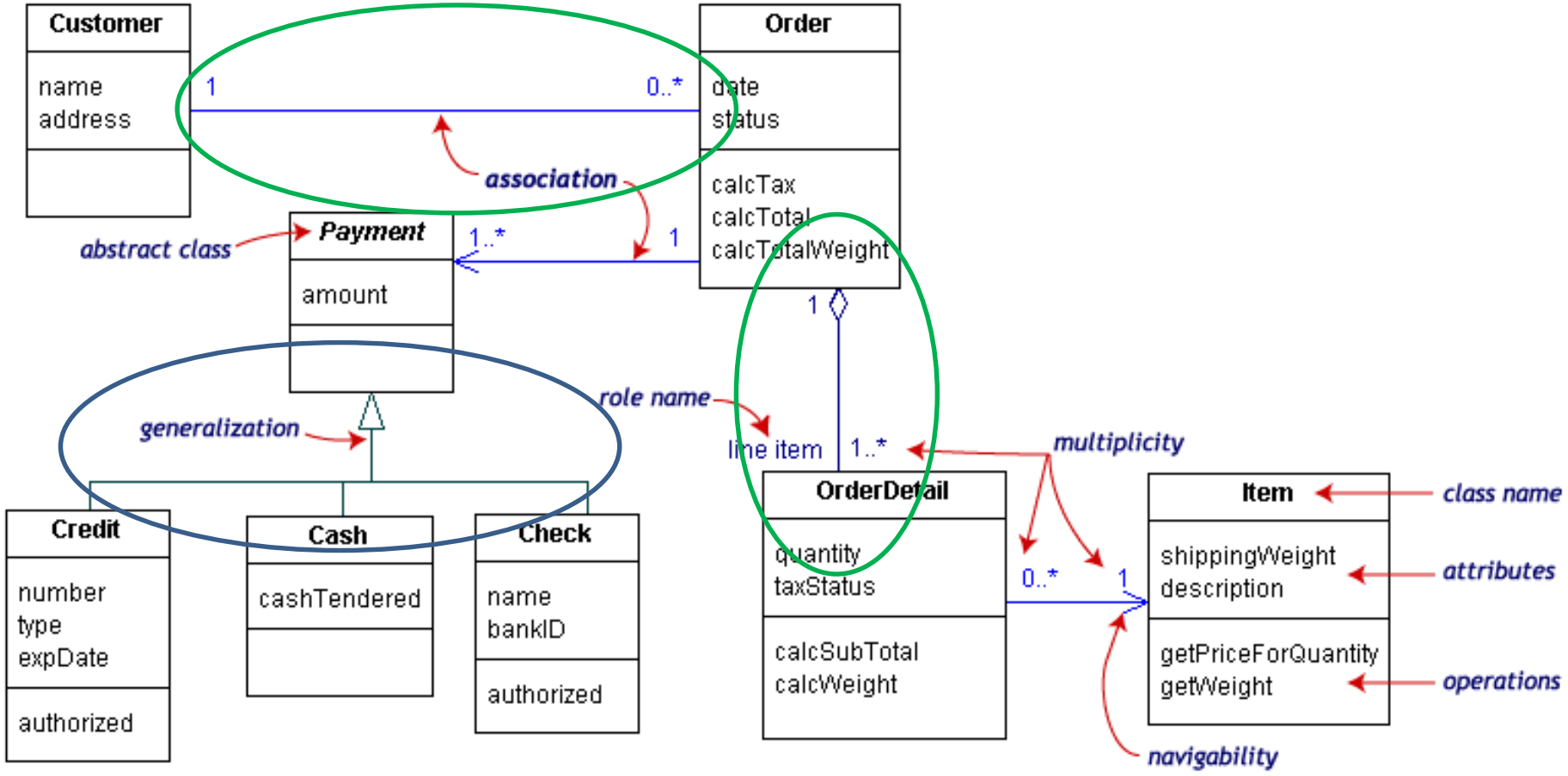
- Alla klasser i Java ärver, direkt eller indirekt, från superklassen ”Object” (sic)
- `public class Account { …`

  är detsamma som
  `public class Account extends Object { …`
- Några ärvda metoder:
  - `public String toString`
  - `public boolean equals(Object other)`
  - `protected Object clone`
  - `public int hashCode()`
- Ovanstående metoder bör normalt omdefinieras i din klass (men vänta med hashCode och clone)
- ”Märk” en omdefinerad metod med ”@Override”

Java:
superklassen
Object



Relationer mellan *objekt* – association, aggregat, komposition
Relationer mellan *klasser* – arv



Källa: http://edn.embarcadero.com/article/images/31863/classdiagramno3d.gif

Nyckelordet final vid arv

- En klass deklarerad final kan inte ärvas, den är slutgiltig
- Klassen String är slutgiltig
  - `public final class String {`
    `…`
- En metod deklarerad final kan inte omdefinieras i subklasser
- Exempel: Vi vill kunna ärva, och utöka/förändra ChessAlgoritm men getFirstPlayer ska alltid vara densamma:
  -
```
class ChessAlgorithm {
    enum ChessPlayer { WHITE, BLACK }
    ...
    public final ChessPlayer getFirstPlayer() {
       return ChessPlayer.WHITE;
    }
    ...
}
```